



正点原子公司名称 : 广州市星翼电子科技有限公司

原子哥在线教学平台 : www.yuanzige.com

开源电子网 / 论坛 : <http://www.openedv.com/forum.php>

正点原子淘宝店铺 : <https://openedv.taobao.com>

正点原子官方网站 : www.alientek.com

正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请关注正点原子公众号, 资料发布更新我们会通知。

请下载原子哥 APP, 数千讲视频免费学习, 更快更流畅。



扫码关注正点原子公众号



扫码下载“原子哥”APP

FIT Image

1.1 概述

RK 平台的 U-Boot 固件有两种格式, RK 格式和 FIT 格式, RK 格式固件和 FIT 格式固件分别由闭源 SPL 和开源 SPL 负责引导。不过对于我们的 RK3568, 我们使用 FIT 格式。

- RK 格式

Rockchip自定义的固件格式, U-Boot和trust分别打包为uboot.img和trust.img。如下:

uboot.img 和32位 trust.img 镜像文件的magic为“LOADER”

```
00000000  4c 4f 41 44 45 52 20 20 00 00 00 00 00 00 00 00 | LOADER
.....|
00000010  00 00 20 00 78 d0 0f 00 06 99 c2 a8 20 00 00 00 | .. .x.....
...|
00000020  09 8a b0 e1 89 7a c2 89 0d e8 da ef 86 3e f2 24
| .....z.....>.$|
```

64位 trust.img 镜像文件的magic为“BL3X”

```
00000000  42 4c 33 58 00 01 00 00 23 00 00 00 f8 00 04 00
| BL3X....#.....|
00000010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
| .....|
```

- FIT 格式

U-Boot mainline支持的一种灵活性极高的固件格式。U-Boot、trust以及mcu等固件一起打包为uboot.img。

uboot.img 的镜像文件的magic 为"d0 0d fe ed", 用命令 `fdtdump uboot.img` 可以查看固件头。

```
00000000  d0 0d fe ed 00 00 06 00 00 00 00 58 00 00 04 c4
| .....X....|
00000010  00 00 00 28 00 00 00 11 00 00 00 10 00 00 00 00 | ...
(| .....|
```

trust 镜像就是 ARM Trusted Firmware + OP-TEE OS 镜像, 对于 RK 平台来说就是 BL31、BL32。

FIT 镜像就是 U-Boot 支持的一种新固件类型的引导镜像格式, 支持任意多个 image 打包和校验。FIT 镜像的制作需要使用 `mkimage` 工具, 该工具在 u-boot 源码目录下的 `tools` 目录中, 不过由于 U-Boot 官方原本的 FIT 功能无法满足实际产品需要, 所以 RK 平台对 FIT 功能进行了适配和优化, 所以自然对 `mkimage` 工

具的源代码进行了修改、优化; 所以对于 RK 平台硬件, 如果使用 FIT 格式镜像, 必须使用 RK U-boot 源码编译生成的 mkimage 工具, 不可使用 U-boot 原版的 mkimage 工具。

FIT 固件会使用 its (image source file) 文件描述 image 信息, 最后通过 mkimage 工具生成 itb (flattened image tree blob) 镜像。its 文件使用 DTS (设备树) 的语法规则, 非常灵活, 可以直接使用 libfdt 库和相关工具。描述文件为 u-boot.its, 最终生成的 FIT 固件为 uboot.itb, 这个 itb 文件其实就是我们的 uboot.img。

当我们编译 u-boot 源码之后, 会在 u-boot 源码目录下生成一个 fit 目录, 该目录下包含 u-boot.its 文件以及编译得到的 uboot.itb 文件。

如下以 u-boot.its 和 u-boot.itb 作为范例进行介绍。

- `/images` : 静态定义了所有的资源, 相当于一个 dtsi 文件;
- `/configurations` : 每个 config 节点都描述了一套可启动的配置, 相当于一个板级 dts 文件。
- `default =` : 指明默认启用的 config;

```
/dts-v1/;

/ {
    description = "FIT Image with ATF/OP-TEE/U-Boot";
    #address-cells = <1>;

    images {
        uboot {
            description = "U-Boot(64-bit)";
            data = /incbin("/.u-boot-nodtb.bin");
            type = "standalone";
            arch = "arm64";
            os = "U-Boot";
            compression = "none";
            load = <0x00a00000>;
            hash {
                algo = "sha256";
            };
        };
        atf-1 {
            description = "ARM Trusted Firmware";
            data = /incbin("/.bl31_0x00040000.bin");
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0x00040000>;
            hash {
                algo = "sha256";
            };
        };
    };
};
```

```
};
atf-2 {
    description = "ARM Trusted Firmware";
    data = /incbin(/".bl31_0xfdcc9000.bin");
    type = "firmware";
    arch = "arm64";
    os = "arm-trusted-firmware";
    compression = "none";
    load = <0xfdcc9000>;
    hash {
        algo = "sha256";
    };
};
atf-3 {
    description = "ARM Trusted Firmware";
    data = /incbin(/".bl31_0xfdc00000.bin");
    type = "firmware";
    arch = "arm64";
    os = "arm-trusted-firmware";
    compression = "none";
    load = <0xfdc00000>;
    hash {
        algo = "sha256";
    };
};
optee {
    description = "OP-TEE";
    data = /incbin(/".tee.bin");
    type = "firmware";
    arch = "arm64";
    os = "op-tee";
    compression = "none";
    load = <0x8400000>;
    hash {
        algo = "sha256";
    };
};
fdt {
    description = "U-Boot dtb";
    data = /incbin(/".u-boot.dtb");
    type = "flat_dt";
    arch = "arm64";
    compression = "none";
```

```

        hash {
            algo = "sha256";
        };
    };

    //configurations 节点下可以定义任意多个不同的 conf 节点,但实际产品方案上我们只需要一个 conf
    即可。
    configurations {
        default = "conf";
        conf {
            description = "rk3568-evb";
            rollback-index = <0x0>;
            firmware = "atf-1";
            loadables = "uboot", "atf-2", "atf-3", "optee";

            fdt = "fdt";
            signature {
                algo = "sha256,rsa2048";
                key-name-hint = "dev";
                sign-images = "fdt", "firmware", "loadables";
            };
        };
    };
};

```

首先需要说明的是, its 文件的语法和设备树文件 dts 是一样的, 这点不用怀疑。只是说 its 文件中的这些属性具体是什么等等这些, 我们需要了解下, 关于 its 文件在 u-boot 源码目录下的 doc 目录中提供了相应的帮助文档:

doc/uImage.FIT/

/images 节点描述定义了所有镜像资源, 对这些镜像资源进行描述, 相当于一个 dtsi 文件, 该节点下定义的镜像并不一定会加入到最终生成的 itb 镜像中。

/configurations: 该节点下可 conf 节点, 每个 conf 节点都描述了一套可用于启动的配置, 相当于一个板级的 dts 文件。

关于这些属性的介绍可以参考 doc/uImage.FIT/source_file_format.txt 文档。

- 根节点的布局:

```
/ o image-tree
|- description = "image description"
|- timestamp = <12399321>
|- #address-cells = <1>
|
o images
|
| o image@1 {...}
| o image@2 {...}
| ...
|
o configurations
|- default = "conf@1"
|
| o conf@1 {...}
| o conf@2 {...}
| ...
```

● /images 节点的布局:

```
o image@1
|- description = "component sub-image description"
|- data = /incbin/("path/to/data/file.bin")
|- type = "sub-image type name"
|- arch = "ARCH name"
|- os = "OS name"
|- compression = "compression name"
|- load = <00000000>
|- entry = <00000000>
|
o hash@1 {...}
o hash@2 {...}
...
```

type 属性: 镜像的类型, 支持的类型有:

```
static const table_entry_t uimage_type[] = {
    { IH_TYPE_AISIMAGE, "aisimage", "Davinci AIS image", },
    { IH_TYPE_FILESYSTEM, "filesystem", "Filesystem Image", },
    { IH_TYPE_FIRMWARE, "firmware", "Firmware", },
    { IH_TYPE_FLATDT, "flat_dt", "Flat Device Tree", },
    { IH_TYPE_GPIMAGE, "gpimage", "TI Keystone SPL Image", },
    { IH_TYPE_KERNEL, "kernel", "Kernel Image", },
    { IH_TYPE_KERNEL_NOLOAD, "kernel_noload", "Kernel Image (no loading done)", },
    { IH_TYPE_KWBIMAGE, "kwbimage", "Kirkwood Boot Image", },
    { IH_TYPE_IMXIMAGE, "imximage", "Freescale i.MX Boot Image", },
    { IH_TYPE_INVALID, "invalid", "Invalid Image", },
    { IH_TYPE_MULTI, "multi", "Multi- File Image", },
    { IH_TYPE_OMAPIMAGE, "omapimage", "TI OMAP SPL With GP CH", },
    { IH_TYPE_PBLIMAGE, "pblimage", "Freescale PBL Boot Image", },
    { IH_TYPE_RAMDISK, "ramdisk", "RAMDisk Image", },
    { IH_TYPE_SCRIPT, "script", "Script", },
    { IH_TYPE_SOCFPGAIMAGE, "socfpgaimage", "Altera SOCFPGA preloader", },
    { IH_TYPE_STANDALONE, "standalone", "Standalone Program", },
    { IH_TYPE_UBLIMAGE, "ublimage", "Davinci UBL image", },
    { IH_TYPE_MXSIMAGE, "mxsimage", "Freescale MXS Boot Image", },
    { IH_TYPE_ATMELIMAGE, "atmelimage", "ATMEL ROM- Boot Image", },
    { IH_TYPE_X86_SETUP, "x86_setup", "x86 setup.bin", },
    { IH_TYPE_LPC32XXIMAGE, "lpc32xximage", "LPC32XX Boot Image", },
    { IH_TYPE_RKIMAGE, "rkimage", "Rockchip Boot Image", },
    { IH_TYPE_RKSD, "rksd", "Rockchip SD Boot Image", },
    { IH_TYPE_RKSPI, "rkspi", "Rockchip SPI Boot Image", },
    { IH_TYPE_VYBRIDIMAGE, "vybridimage", "Vybrid Boot Image", },
    { IH_TYPE_ZYNQIMAGE, "zynqimage", "Xilinx Zynq Boot Image", },
    { IH_TYPE_ZYNQMPIMAGE, "zynqmpimage", "Xilinx ZynqMP Boot Image", },
    { IH_TYPE_FPGA, "fpga", "FPGA Image", },
    { IH_TYPE_TEE, "tee", "Trusted Execution Environment Image", },
    { IH_TYPE_FIRMWARE_IVT, "firmware_ivt", "Firmware with HABv4 IVT", },
    { IH_TYPE_PMMC, "pmmc", "TI Power Management Micro- Controller Firmware", },
    { IH_TYPE_RKNAND, "rk NAND", "Rockchip NAND Boot Image", },
    { -1, "", "", },
};
```

data 属性: 二进制数据（镜像文件）的路径;

compression 属性: 镜像数据的压缩类型，支持的压缩是“gzip”和“bzip2”。如果不使用压缩，则压缩属性应设置为“none”。

```
static const table_entry_t uimage_comp[] = {
    { IH_COMP_NONE, "none", "uncompressed", },
    { IH_COMP_BZIP2, "bzip2", "bzip2 compressed", },
    { IH_COMP_GZIP, "gzip", "gzip compressed", },
    { IH_COMP_LZMA, "lzma", "lzma compressed", },
    { IH_COMP_LZO, "lzo", "lzo compressed", },
    { IH_COMP_LZ4, "lz4", "lz4 compressed", },
    { -1, "", "", },
};
```

以上三个是必填属性: type、data、compression。

接下来这些为<有条件的强制性属性>:

os 属性: OS name, type 等于"kernel"和"ramdisk"时必须填写, 包括:

```
static const table_entry_t uimage_os[] = {
    { IH_OS_INVALID, "invalid", "Invalid OS", },
    { IH_OS_OP_TEE, "op-tee", "OP-TEE", },
    { IH_OS_ARM_TRUSTED_FIRMWARE, "arm-trusted-firmware", "ARM Trusted Firmware", },
    { IH_OS_LINUX, "linux", "Linux", },
    #if defined(CONFIG_LYNXKDI) || defined(USE_HOSTCC)
    { IH_OS_LYNXOS, "lynxos", "LynxOS", },
    #endif
    { IH_OS_NETBSD, "netbsd", "NetBSD", },
    { IH_OS_OSE, "ose", "Enea OSE", },
    { IH_OS_PLAN9, "plan9", "Plan 9", },
    { IH_OS_RTEMS, "rtems", "RTEMS", },
    { IH_OS_U_BOOT, "u-boot", "U-Boot", },
    { IH_OS_VXWORKS, "vxworks", "VxWorks", },
    #if defined(CONFIG_CMD_ELF) || defined(USE_HOSTCC)
    { IH_OS_QNX, "qnx", "QNX", },
    #endif
    #if defined(CONFIG_INTEGRITY) || defined(USE_HOSTCC)
    { IH_OS_INTEGRITY, "integrity", "INTEGRITY", },
    #endif
    #ifdef USE_HOSTCC
    { IH_OS_4_4BSD, "4_4bsd", "4_4BSD", },
    { IH_OS_DELL, "dell", "Dell", },
    { IH_OS_ESIX, "esix", "Esix", },
    { IH_OS_FREEBSD, "freebsd", "FreeBSD", },
    { IH_OS_IRIX, "irix", "Irix", },
    { IH_OS_NCR, "ncr", "NCR", },
    { IH_OS_OPENBSD, "openbsd", "OpenBSD", },
    { IH_OS_PSOS, "psos", "pSOS", },
    { IH_OS_SCO, "sco", "SCO", },
    { IH_OS_SOLARIS, "solaris", "Solaris", },
    { IH_OS_SVR4, "svr4", "SVR4", },
    #endif
    #if defined(CONFIG_BOOTM_OPENRTOS) || defined(USE_HOSTCC)
    { IH_OS_OPENRTOS, "openrtos", "OpenRTOS", },
    #endif
};
```

arch 属性: 架构名字, type 等于 "standalone", "kernel", "firmware", "ramdisk" and "fdt"时必须填写, 支持:

```
static const table_entry_t uimage_arch[] = {
    { IH_ARCH_INVALID, "invalid", "Invalid ARCH", },
    { IH_ARCH_ALPHA, "alpha", "Alpha", },
    { IH_ARCH_ARM, "arm", "ARM", },
    { IH_ARCH_I386, "x86", "Intel x86", },
    { IH_ARCH_IA64, "ia64", "IA64", },
    { IH_ARCH_M68K, "m68k", "M68K", },
    { IH_ARCH_MICROBLAZE, "microblaze", "MicroBlaze", },
    { IH_ARCH_MIPS, "mips", "MIPS", },
    { IH_ARCH_MIPS64, "mips64", "MIPS 64 Bit", },
    { IH_ARCH_NIOS2, "nios2", "NIOS II", },
    { IH_ARCH_PPC, "powerpc", "PowerPC", },
    { IH_ARCH_PPC, "ppc", "PowerPC", },
    { IH_ARCH_S390, "s390", "IBM S390", },
    { IH_ARCH_SH, "sh", "SuperH", },
    { IH_ARCH_SPARC, "sparc", "SPARC", },
    { IH_ARCH_SPARC64, "sparc64", "SPARC 64 Bit", },
    { IH_ARCH_BLACKFIN, "blackfin", "Blackfin", },
    { IH_ARCH_AVR32, "avr32", "AVR32", },
    { IH_ARCH_NDS32, "nds32", "NDS32", },
    { IH_ARCH_OPENRISC, "or1k", "OpenRISC 1000", },
    { IH_ARCH_SANDBOX, "sandbox", "Sandbox", },
    { IH_ARCH_ARM64, "arm64", "AArch64", },
    { IH_ARCH_ARC, "arc", "ARC", },
    { IH_ARCH_X86_64, "x86_64", "AMD x86_64", },
    { IH_ARCH_XTENSA, "xtensa", "Xtensa", },
    { IH_ARCH_RISCV, "riscv", "RISC-V", },
    { -1, "", "" },
};
```

entry 属性: 入口点地址。

load 属性: 加载地址, 其实加载地址就是入口地址, 这俩的含义是一样的我觉得, RK 平台使用 its 文件中并没有使用 entry 属性, 而是用了 load 属性。

“/images”节点下有一个可选的子节点, hash 节点, hash 节点用于做 hash 值校验, 他通常有两个属性 algo、value,

algo 属性: hash 校验算法的名称, supported are "crc32", "md5" and "sha1"。

value 属性: 实际校验和或哈希值, 对应长度为 4、16 或 20 个字节, 编译过程中构建该属性。

- **'/configurations'节点的布局**

```
| - default = "default configuration sub-node unit name"
|
o config@1 {...}
o config@2 {...}
...
```

“/configurations”节点下有一个 default 属性, 表示当前那个配置 (某个 config 节点) 生效。

来看看 config 节点的布局:

```
o config@1
| - description = "configuration description"
| - kernel = "kernel sub-node unit name"
| - ramdisk = "ramdisk sub-node unit name"
| - fdt = "fdt sub-node unit-name" [, "fdt overlay sub-node unit-name", ...]
| - fpga = "fpga sub-node unit-name"
| - loadables = "loadables sub-node unit-name"
```

需要说明的是, 还有一些属性会在 its 编译过程中自动创建出来, 因为这些属性与加载地址、镜像大小等有关系, 譬如 data-offset、data-size、data-position 等这些, 以及一些满足 RK 平台需要的属性 (RK 修改的, 这些属性不是原本 mkimage 工具所支持的)。

itb 文件

使用 mkimage 工具和 its 文件可以生成 itb 文件:

```
[u-boot.its] + [images]      mkimage + dtc      [u-boot.itb]
                             =====>
```

fdtdump 命令可以查看 itb 文件内容, 可以看到

fdtdump fit/uboot.itb

```
//dts-v1/;
// magic: 0xd00dfeed
// totalsize: 0xa00 (2560)
// off_dt_struct: 0x48
// off_dt_strings: 0x7f8
// off_mem_rsvmap: 0x28
// version: 17
// last_comp_version: 16
// boot_cpuid_phys: 0x0
// size_dt_strings: 0xc5
// size_dt_struct: 0x7b0

/memreserve/ 0x7ff83d4de000 0xa00;
/ {
    version = <0x00000000>;
    totalsize = <0x001bf600>;
    timestamp = <0x616cd543>;
    description = "FIT Image with ATF/OP-TEE/U-Boot";
    #address-cells = <0x00000001>;
    images {
        uboot {
            data-size = <0x00122060>;
            data-position = <0x00000e00>;
            description = "U-Boot(64-bit)";
            type = "standalone";
            arch = "arm64";
            os = "U-Boot";
            compression = "none";
            load = <0x00a00000>;
            hash {
                value = <0xa424695a 0xa5b7160b 0xcdfbab8 0xd2ec208a 0x3d0397c0 0x913002a1 0x9fe8fd11 0xafcaeeea>;
                algo = "sha256";
            };
        };
        atf-1 {
            data-size = <0x00029743>;
            data-position = <0x00123000>;
            description = "ARM Trusted Firmware";
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0x00040000>;
            hash {
                value = <0xee4ec368 0xc58f9c20 0x766a2b3c 0x41198537 0xbaeb82b5 0xd0646bfa 0xe68498a4 0x2fbb98f7>;
                algo = "sha256";
            };
        };
        atf-2 {
            data-size = <0x00002000>;
            data-position = <0x0014c800>;
            description = "ARM Trusted Firmware";
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0xfdc00000>;
            hash {
                value = <0xf1fecab9 0x719665ae 0xba73361d 0x8709306b 0x8fe308ca 0x13f28515 0x4d32833f 0x5d0b68b5>;
                algo = "sha256";
            };
        };
        atf-3 {
            data-size = <0x00001d68>;
            data-position = <0x0014e800>;
            description = "ARM Trusted Firmware";
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0xfdc00000>;
            hash {
                value = <0xd7aa45eb 0x185009e9 0xa6d0c24b 0x9a88f31c 0xad489f3f 0x6335933a 0xb3007d86 0xb3c372d>;
                algo = "sha256";
            };
        };
        optee {
            data-size = <0x0006b390>;
            data-position = <0x00150600>;
            description = "OP-TEE";
            type = "firmware";
            arch = "arm64";
            os = "op-tee";
            compression = "none";
            load = <0x08400000>;
            hash {
                value = <0xb8cddafa 0xb0c4e20b 0x23a2eec4 0xc41bdb25 0x2325b693 0x33c07f2d 0x5a174428 0x0468c850>;
                algo = "sha256";
            };
        };
    };
};
```

```

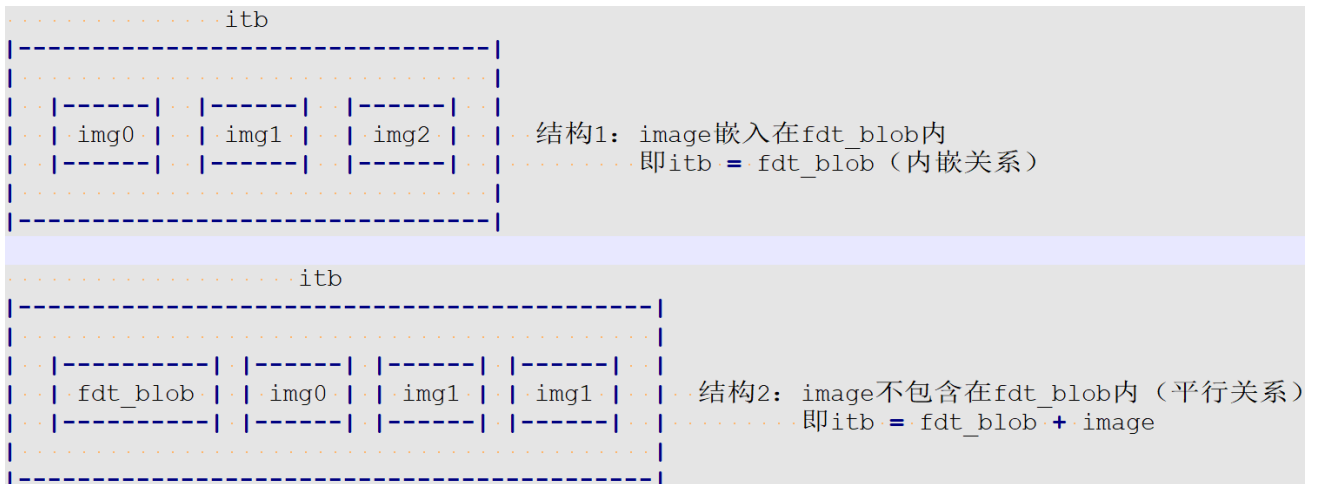
fdt {
    data-size = <0x000037b9>;
    data-position = <0x001bba00>;
    description = "U-Boot dtb";
    type = "flat dt";
    arch = "arm64";
    compression = "none";
    hash {
        value = <0x9679fe2a 0x45776f35 0x494f95cc 0xbf57831f 0x94c54cc9 0x0b3766a2 0xf7371e16 0x9ac928fb>;
        algo = "sha256";
    };
};

};
configurations {
    default = "conf";
    conf {
        description = "rk3568-evb";
        rollback-index = <0x00000000>;
        firmware = "atf-1";
        loadables = "uboot", "atf-2", "atf-3", "optee";
        fdt = "fdt";
        signature {
            algo = "sha256,rsa2048";
            key-name-hint = "dev";
            sign-images = "fdt", "firmware", "loadables";
        };
    };
};
};

```

这里面有些属性就是编译之后才会有的,编译的时候创建的属性,譬如 data-size、data-offset、data-position 等等,这些属性后面再说。

itb 本质是 fdt_blob (its 设备树镜像) + images 的文件集合,有如下两种打包方式,RK 平台方案采用结构 2 方式。



使用 mkimage 工具时,默认使用的是第一种结构生成 itb,可通过 -E 选项,指定为第二种结构来生成 itb。

对于结构 2,u-boot 以及 FIT 文档中也称为“External data”,外部数据,也就是说镜像没有嵌入到 fdt_blob,而是在外部。这个时候通常会使用 data-offset 或者 data-position 属性来找到该镜像的位置,data-size 表示镜像数据的大小。

那么 data-offset 和 data-position,一个表示相对位置,一个表示绝对位置。不过 data-position 的优先级会更高。默认情况下使用的是 data-offset 属性,不过我们可以使用 -p 选项指定为使用绝对位置 data-position,使用 -p 选项时,后面可以携带一个参数,这个参数指定了一个起始位置,所有的“External data”镜像将从这个位置开始放置。

data-offset: 相对位置,这个是镜像相对于 fdt_blob 末尾的位置偏移量,所以相对整个 itb 文件的位置偏移量需要加上 fdt_blob 的大小。

data-position: 绝对位置,镜像在 itb 文件中的绝对位置(其实就是相对于 itb 文件头部的位置)。

u-boot 在解析 itb (uboot.img 镜像的时候,不过 uboot.img 通过由 SPL 加载解析),会找到镜像对应的节点,也就是“/images”节点下定义的某个镜像的节点,再找到它的这些属性,根据这些属性找到它的位置。

在解析 fdt_blob 之前, SPL 会将 fdt_blob 拷贝到内存中, 这个在 SPL 启动流程中分析过, 然后在解析, 找到位置之后再从存储设备中读取。

如果结构 1 这种内嵌的情况, 通过会通过 data 属性来找到, 找到了 data 属性就找到镜像的位置, 因为结构 2 是内嵌的情况, 镜像已经嵌入到了 fdt_blob 中, 所以拷贝 fdt_blob 的时候就已经将这些镜像都拷贝到内存中了。但如果没有 data 属性, 则还会通过 data-size、data-offset、或 data-position 这些属性定位, 具体看 SPL 代码分析过程。

再次说明, itb 文件就是我们编译的到的 uboot.img 文件。

- uboot.img 文件

uboot.itb = trust + u-boot.bin + mcu.bin(option)

uboot.img = uboot.itb * N份 (N一般是2)

trust 和 mcu 文件来自rkbin工程, 编译脚本会自动从rkbin工程索引并获取它们。

itb 制作过程

编译完 u-boot 源码之后, 会生成很多的.bin 文件:

```

alientek@android:~/hdd/rk3568-a/u-boot$
alientek@android:~/hdd/rk3568-a/u-boot$ ls
api          bl31.elf    configs    dts        include    MAINTAINERS  PREUPLOAD.cfg  spl          tpl          u-boot.dtb  u-boot-nodtb.bin
arch         board       disk       env         Kbuild     Makefile     README         System.map   tee.bin      u-boot-dtb.bin  u-boot.srec
bl31_0x00040000.bin  common     doc         examples   Kconfig    make.sh      rk356x_spl_loader_v1.09.111.bin  test        u-boot.bin  u-boot-dtb.bin  u-boot.sym
bl31_0x0fdcc9000.bin  config.mk  drivers    fit         lib         net          scripts         tools       u-boot.cfg  u-boot.lds     u-boot.map
bl31_0x0fdcd0000.bin  config.mk  drivers    fs          licenses   post         snapshot.commit  tools       u-boot.cfg.configs  u-boot.map
alientek@android:~/hdd/rk3568-a/u-boot$

```

可以看到有 bl31_0xxxxx.bin、bl31.elf、tee.bin 等这些镜像文件。

首先这个 bl31.elf 文件就是 rbbin/bin/rk35/目录下的 rk3568_bl31_v1.27.elf 文件, 直接拷贝过来的, tee.bin 文件就是 rbbin/bin/rk35/目录下 rk3568_bl32_v1.05.bin 文件, 也是直接拷贝过来的。

而 bl31_0x00040000.bin、bl31_0x0fdcc9000.bin、bl31_0x0fdcd0000.bin 这几个文件其实只是由 bl31.elf 分解而成的, 在制作 itb 文件的时候, 会对 bl31.elf 文件进行分解, 而实际上制作 itb 文件时, 也是使用了这 3 个分解后的文件和 tee.bin 文件, 在 its 文件中有写到, 如下:

atf1: data = /incbin("/.bl31_0x00040000.bin");

atf2: data = /incbin("/.bl31_0x0fdcc9000.bin");

atf3: data = /incbin("/.bl31_0x0fdcd0000.bin");

optee: data = /incbin("/.tee.bin");

这三个文件通过 arch/arm/mach-rockchip/decode_bl31.py 脚本生成。

后面的数字表示它对应的加载地址, 也就可以知道, 这个 bl31 分成了 3 个部分, 这 3 个部分对应的加载地址是不同的, SPL 会引导 bl31_0x00040000.bin 启动, 后续的就由它去处理了, 最终会启动 u-boot 镜像。

我们说过, uboot.img 其实就包括了 u-boot 镜像 (BL33)、ARM Trusted Firmware (BL31)、OP-TEE OS (BL32), 我们来看看 itb 文件的制作过程:

分析 make.sh 脚本文件:

make.sh 脚本中会调用 pack_images 函数进行打包:

```
function pack_images()
{
    if [ "${ARG_RAW_COMPILE}" != "y" ]; then
        if [ "${PLAT_TYPE}" == "FIT" ]; then
            pack_fit_image ${ARG_LIST_FIT}
        else
            pack_uboot_image
            pack_trust_image
            pack_loader_image
        fi
    fi
}
```

接着调用 pack_fit_image 函数:

```
function pack_fit_image()
{
    # check host tools
    if ! which dtc >/dev/null 2>&1 ; then
        echo "ERROR: No 'dtc', please: apt-get install device-tree-compiler"
        exit 1
    fi

    # If we don't plan to have uboot in uboot.img in case of: SPL => Trust => Kernel, creating empty files.
    if [ "${ARG_NO_UBOOT}" == "y" ]; then
        rm u-boot-nodtb.bin u-boot.dtb -f
        touch u-boot-nodtb.bin u-boot.dtb
    fi

    rm uboot.img trust*.img -rf
    ${SCRIPT_FIT} ${ARG_LIST_FIT} --chip ${RKCHIP_LABEL}

    rm ${REP_DIR} -rf
    echo "pack uboot.img okay! Input: ${INI_TRUST}"
}
```

然后调用 \${SCRIPT_FIT} \${ARG_LIST_FIT} --chip \${RKCHIP_LABEL}

\${SCRIPT_FIT} 就是 ./scripts/fit.sh 脚本文件:

```
if [ ! -z "${ARG_VALIDATE}" ]; then
    validate_arg ${ARG_VALIDATE}
else
    fit_raw_compile
    if [ ! -z "${ARG_RECOVERY_IMG}" ]; then
        fit_gen_recovery_itb
        fit_gen_recovery_img
    fi
    # "--boot_img_dir" is for U-Boot debug only
    if [ ! -z "${ARG_BOOT_IMG}" -o ! -z "${ARG_BOOT_IMG_DIR}" ]; then
        fit_gen_boot_itb
        fit_gen_boot_img
    fi
    fit_gen_uboot_itb
    fit_gen_uboot_img
    fit_gen_loader

    echo
    fit_msg_uboot
    fit_msg_recovery
    fit_msg_boot
    fit_msg_loader
fi
```

依次调用 fit_gen_uboot_itb、fit_gen_uboot_img、fit_gen_loader 这三个函数, 在 fit_gen_uboot_itb 函数会其生成 itb 文件, 如下所示:

```
function fit_gen_uboot_itb()
{
    # generate u-boot.its file
    ./make.sh itb ${ARG_INI_TRUST}

    # check existence of file in its
    check_its ${ITS_UBOOT}

    if [ "${ARG_SIGN}" != "y" ]; then
        ${MKIMAGE} -f ${ITS_UBOOT} -E -p ${OFFS_DATA} ${ITB_UBOOT} -v ${ARG_VER_UBOOT}
        if [ "${ARG_SPL_NEW}" == "y" ]; then
            ./make.sh --spl ${ARG_INI_LOADER}
            echo "pack loader with new: spl/u-boot-spl.bin"
        else
            ./make.sh loader ${ARG_INI_LOADER}
        fi
    else
        if [ ! -f ${RSA_PRI_KEY} ]; then
            echo "ERROR: No ${RSA_PRI_KEY} "
            exit 1
        elif [ ! -f ${RSA_PUB_KEY} ]; then
            echo "ERROR: No ${RSA_PUB_KEY} "
            exit 1
        fi

        if ! grep -q '^CONFIG_SPL_FIT_SIGNATURE=y' .config ; then
            echo "ERROR: CONFIG_SPL_FIT_SIGNATURE is disabled"
            exit 1
        fi
    fi
}
```

./make.sh itb \${ARG_INI_TRUST} 用于产生 u-boot.its 文件。

\${MKIMAGE} -f \${ITS_UBOOT} -E -p \${OFFS_DATA} \${ITB_UBOOT} -v \${ARG_VER_UBOOT} 用于生成 uboot.itb 文件。

然后在 fit_gen_uboot_img 函数中生成 uboot.img 镜像文件, 其实就是对 uboot.itb 文件的拷贝, 可能有多份 itb 文件:

```
function fit_gen_uboot_img()
{
    ITB=$1

    if [ -z ${ITB} ]; then
        ITB=${ITS_UBOOT}
    fi

    ITB_MAX_NUM=`sed -n "/SPL_FIT_IMAGE_MULTIPLE/p" .config | awk -F "=" '{ print $2 }'`
    ITB_MAX_KB=`sed -n "/SPL_FIT_IMAGE_KB/p" .config | awk -F "=" '{ print $2 }'`
    ITB_MAX_BS=$((ITB_MAX_KB*1024))
    ITB_BS=`ls -l ${ITB} | awk '{ print $5 }'`

    if [ ${ITB_BS} -gt ${ITB_MAX_BS} ]; then
        echo "ERROR: pack ${IMG_UBOOT} failed! ${ITB} actual: ${ITB_BS} bytes, max limit: ${ITB_MAX_BS} bytes"
        exit 1
    fi

    rm -f ${IMG_UBOOT}
    for ((i = 0; i < ${ITB_MAX_NUM}; i++));
    do
        cat ${ITB} >> ${IMG_UBOOT}
        truncate -s ${ITB_MAX_KB}K ${IMG_UBOOT}
    done
}
```